Webserv

Bu, URL'lerin neden HTTP ile başladığını sonunda anladığın zamandır

Özet:

Bu proje, kendi HTTP server'ınızı yazmanız hakkındadır. Bunu gerçek bir browser ile test edebileceksiniz. HTTP, internet üzerinde en yaygın olarak kullanılan protokollerden biridir. Bir website üzerinde çalışmıyor olsanız bile, HTTP'nin karmaşıklıklarını anlamak faydalı olacaktır.

Sürüm: 24.0

İçindekiler

I Giriş II Genel Kurallar III AI Talimatları IV Zorunlu Kısım IV.1 Gereksinimler IV.2 Sadece MacOS İçin IV.3 Configuration Dosyası V README Gereksinimleri VI Bonus Kısım VII Gönderim ve Eş Değerlendirme

---

Bölüm I: Giriş

Hypertext Transfer Protocol (HTTP), dağıtılmış, işbirliğine dayalı hypermedia bilgi sistemleri için bir application protokolüdür.

HTTP, World Wide Web için veri iletişiminin temelini oluşturur; burada hypertext belgeler, kullanıcıların kolayca erişebileceği diğer kaynakların bağlantılarını içerir. Örneğin, mouse düğmesine tıklamak veya web browser'da ekrana dokunmak yoluyla.

HTTP, hypertext işlevselliğini desteklemek ve World Wide Web'in büyümesini sağlamak için geliştirilmiştir.

Bir web server'ının temel işlevi, web sayfalarını depolama, işleme ve istemcilere teslim etmektir. İstemci-sunucu iletişimi, Hypertext Transfer Protocol (HTTP) aracılığıyla gerçekleşir.

Teslim edilen sayfalar en sık olarak HTML belgeleridir; bunlar metin içeriğinin yanı sıra görüntüler, stil sayfaları ve script'ler içerebilir.

Yüksek trafikli bir website için birden fazla web server kullanılabilir; traffic birden fazla fiziksel makinaya bölünür.

Bir user agent (genellikle web browser veya web crawler), HTTP kullanarak belirli bir kaynağı talep ederek iletişimi başlatır ve server, o kaynağın içeriğini veya bunu yapamazsa bir hata mesajını yanıt olarak gönderir. Kaynak, tipik olarak server'ın depolamasında gerçek bir dosya veya bir program'ın sonucu olup, her zaman durum böyle olmaz ve birçok başka şey de olabilir.

HTTP'nin birincil işlevi content sunmak olsa da, istemcilerin veri göndermesini de sağlar. Bu özellik web formlarını göndermek ve dosya yükleme için kullanılır.

Bölüm II: Genel Kurallar

- Programınız hiçbir koşulda çökmemelidir (bellek tükense bile) veya beklenmedik şekilde sonlanmamalıdır. Bunun olması durumunda projeniz işlevsel olmayan olarak kabul edilir ve notunuz 0 olur.
- Kaynak dosyalarınızı compile eden bir Makefile göndermelisiniz. Gereksiz relinking yapmaz.
- Makefile'ınız en azından şu kuralları içermelidir: $(NAME), all, clean, fclean ve re.
- Kodunuzu c++ ve -Wall -Wextra -Werror flags'leri ile compile edin.
- Kodunuz C++ 98 standardına uymalı ve -std=c++98 flag'i eklendiğinde de compile olmaya devam etmelidir.
- Mümkün olduğunca birçok C++ özelliğini kullanmaya çalışın (örneğin, <string.h> yerine <cstring> seçin). C fonksiyonlarını kullanmanıza izin verilir, ancak mümkün olduğunca her zaman C++ versiyonlarını tercih edin.
- Hiçbir external library ve Boost library'si yasaktır.

Bölüm III: AI Talimatları

Bağlam

Öğrenme yolculuğunuzda, AI birçok farklı görevde size yardımcı olabilir. AI araçlarının çeşitli yeteneklerini ve çalışmanızı nasıl destekleyebileceğini keşfetmek için zaman ayırın. Ancak, her zaman ihtiyatlı davranın ve sonuçları eleştirel olarak değerlendirin. Code, dokümantasyon, fikirler veya teknik açıklamalar olsun, sorunuzun iyi şekilde formüle edilip edilmediğinden veya oluşturulan içeriğin doğru olup olmadığından asla tamamen emin olamazsınız. Akranlarınız, hataları ve kör noktaları önlemenize yardımcı olmak için değerli bir kaynaktır.

Ana Mesaj

➠ Tekrarlayan veya sıkıcı görevleri azaltmak için AI kullanın.

➠ Kodlama ve non-kodlama prompting becerilerini geliştirin — bunlar gelecekteki kariyerinize fayda sağlayacaktır.

➠ AI sistemlerinin nasıl çalıştığını öğrenin; böylece yaygın riskleri, önyargıları ve etik sorunları daha iyi tahmin edip önleyebilirsiniz.

➠ Akranlarınızla çalışarak hem teknik hem de power skills geliştirmeye devam edin.

➠ Sadece tamamen anladığınız ve sorumluluğunu üstlenebileceğiniz AI tarafından üretilen içeriği kullanın.

Öğrenci Kuralları:

- AI araçlarını keşfetmek ve nasıl çalıştığını anlamak için zaman ayırmalısınız; böylece bunları etik bir şekilde kullanabilir ve olası önyargıları azaltabilirsiniz.
- Prompt yazmadan önce probleminiz üzerine düşünmelisiniz — bu, daha net, daha ayrıntılı ve daha uygun prompts yazmanıza yardımcı olur.
- AI tarafından oluşturulan herhangi bir şeyi sistematik olarak kontrol etme, inceleme, sorgulama ve test etme alışkanlığı geliştirmelisiniz.
- Her zaman eş değerlendirmesi aramalısınız — sadece kendi doğrulamanıza güvenmeyin.

Aşama Çıktıları:

- Genel amaçlı ve alana özgü prompting becerilerini geliştirin.
- AI araçlarının etkili kullanımıyla üretkenliğinizi artırın.
- Computational thinking, problem çözme, uyarlanabilirlik ve işbirliğini güçlendirmeye devam edin.

Yorumlar ve Örnekler:

- Düzenli olarak — sınavlar, değerlendirmeler ve daha fazlası — gerçek anlayış göstermeniz gereken durumlarla karşılaşacaksınız. Hazırlıklı olun; hem teknik hem de kişilerarası becerilerinizi geliştirmeye devam edin.
- Akranlarınızla mantığınızı açıklamak ve tartışmak, genellikle anlayışınızdaki boşlukları ortaya çıkarır. Eş öğrenimini bir öncelik yapın.
- AI araçları sıklıkla spesifik bağlamınızdan yoksun olur ve genel yanıtlar sağlamaya eğilimlidir. Ortamınızı paylaşan akranlarınız daha alakalı ve doğru içgörüler sağlayabilir.
- AI en olası cevabı üretme eğilimindeyken, akranlarınız alternatif perspektifler ve değerli nüans sağlayabilir. Onlara bir kalite kontrol noktası olarak güvenin.

✓ İyi Uygulama: AI'ye "Bir sorting fonksiyonunu nasıl test ederim?" diye sordum. Bana birkaç fikir verdi. Bunları denedim ve sonuçları bir akranla gözden geçirdim. Yaklaşımı birlikte geliştirdik.

✗ Kötü Uygulama: AI'den bütün bir fonksiyon yazmasını istedim, projeme copy-paste ettim. Eş değerlendirmesi sırasında ne yaptığını veya neden yaptığını açıklayamadım. İtibarımı kaybettim — ve projemi başarısız buldum.

✓ İyi Uygulama: Bir parser tasarlamaya yardımcı olmak için AI kullandım. Ardından mantığı bir akranla gözden geçirdim. İki bug yakaladık ve birlikte yeniden yazdık — daha iyi, daha temiz ve tamamen anlaşıldı.

✗ Kötü Uygulama: Copilot'un projelerimde önemli bir kısmı için code oluşturmasına izin verdim. Compile oldu, ama pipe'ları nasıl işlediğini açıklayamadım. Değerlendirme sırasında gerekçesi gösteremedim ve projemi başarısız buldum.

Bölüm IV: Zorunlu Kısım

Program Adı: webserv Gönderilecek Dosyalar: Makefile, *.{h, hpp}, *.cpp, *.tpp, *.ipp, configuration dosyaları Makefile: NAME, all, clean, fclean, re Argümanlar: [Bir configuration dosyası] External Fonksiyonlar: Tüm fonksiyonalite C++ 98'de uygulanmalıdır. execve, pipe, strerror, gai_strerror, errno, dup, dup2, fork, socketpair, htons, htonl, ntohs, ntohl, select, poll, epoll (epoll_create, epoll_ctl, epoll_wait), kqueue (kqueue, kevent), socket, accept, listen, send, recv, chdir, bind, connect, getaddrinfo, freeaddrinfo, setsockopt, getsockname, getprotobyname, fcntl, close, read, write, waitpid, kill, signal, access, stat, open, opendir, readdir ve closedir. Libft İzinli: Yok Açıklama: C++ 98'de bir HTTP server

Executable'ı şu şekilde çalıştırmalıdır:

./webserv [configuration dosyası]

NOT: Konu ve değerlendirme sayfasında poll() belirtilmiş olsa da, select(), kqueue() veya epoll() gibi eşdeğer herhangi bir işlevi kullanabilirsiniz.

IV.1 Gereksinimler

UYARI: HTTP protokolünü tanımlayan RFC'leri okuyun ve bu projeyi başlamadan önce telnet ve NGINX ile testler yapın. Tüm RFC'leri uygulamanız gerekmemekle birlikte, okuması gerekli özellikleri geliştirmenize yardımcı olacaktır. HTTP 1.0, referans nokta olarak önerilir ancak zorunlu değildir.

- Programınız, command line'da argument olarak sağlanan bir configuration dosyası kullanmalı veya default path'de bulunan dosyayı kullanmalıdır.
- Başka bir web server'ı execve edemezsiniz.
- Server'ınız her zaman non-blocking kalmalı ve gerektiğinde client bağlantı kopması durumunu doğru şekilde işlemelidir.
- Non-blocking olmalı ve istemciler ile server arasındaki tüm I/O işlemleri için sadece 1 poll() (veya eşdeğer) kullanmalıdır (listen dahil).
- poll() (veya eşdeğer) aynı anda hem okuma hem de yazma işlemlerini izlemelidir.
- poll() (veya eşdeğer) üzerinden gitmeden asla read veya write işlemi yapmayın.
- Read veya write işlemi yaptıktan sonra errno değerini kontrol ederek server davranışını ayarlamak kesinlikle yasaktır.

- Regular disk dosyaları için poll() (veya eşdeğer bir işlev) kullanmanız gerekmez; bunlardaki read() ve write() readiness notifications gerektirmez.

UYARI: Veri bekleme durumunda olabilecek I/O (socket'ler, pipe'lar/FIFO'lar, vb.) non-blocking olmalı ve tek bir poll() (veya eşdeğer) tarafından kontrol edilmelidir. Bu descriptor'lar üzerinde önceden readiness kontrolü olmadan read/recv veya write/send çağrısı yapılması 0 notu ile sonuçlanır. Regular disk dosyaları bu kuralın dışındadır.

- poll() veya eşdeğer bir çağrı kullanırken, ilişkili tüm macro'ları veya helper fonksiyonlarını kullanabilirsiniz (örneğin, select() için FD_SET).
- Server'ınıza gelen bir istek hiçbir zaman sonsuza kadar askıda kalmamalıdır.
- Server'ınız, seçtiğiniz standart web browser'larla uyumlu olmalıdır.
- NGINX, header'ları karşılaştırmak ve davranış yanıtlarını kontrol etmek için kullanılabilir (HTTP sürümleri arasındaki farklılıklara dikkat edin).
- HTTP response status kodlarınız doğru olmalıdır.
- Hiç biri sağlanmazsa server'ınız default error page'lerine sahip olmalıdır.
- fork'u CGI dışında (PHP veya Python gibi) herhangi bir şey için kullanamazsınız.
- Tamamen static bir website sunmaya hazır olmalısınız.
- İstemciler dosya yükleme yapabilmelidir.
- En azından GET, POST ve DELETE method'larına ihtiyacınız vardır.
- Server'ınızı stres testine tabi tutun; her zaman mevcut kalması gerekir.
- Server'ınız farklı content sunmak için birden fazla port'ta dinleyebilmelidir (Configuration dosyasına bakın).

NOT: HTTP RFC'nin yalnızca bir subset'ini sunmak için kasıtlı olarak seçim yaptık. Bu bağlamda, virtual host özelliği kapsam dışında kabul edilir. Ancak, isterseniz uygulamak için izniniz var.

IV.2 Sadece MacOS İçin

NOT: macOS, write()'i diğer Unix tabanlı işletim sistemlerinden farklı şekilde işlediğinden, fcntl() kullanmanıza izin verilir. Diğer Unix işletim sistemlerine benzer davranış sağlamak için file descriptor'ları non-blocking mode'da kullanmalısınız.

UYARI: Ancak, fcntl()'i yalnızca aşağıdaki flag'lerle kullanabilirsiniz: F_SETFL, O_NONBLOCK ve FD_CLOEXEC. Başka herhangi bir flag yasaktır.

IV.3 Configuration Dosyası

İPUÇU: NGINX configuration dosyasının 'server' bölümünden ilham alabilirsiniz.

Configuration dosyasında aşağıdakileri yapabilmelisiniz:

- Server'ınızın dinleyeceği tüm interface:port çiftlerini tanımlayın (program tarafından sunulan birden fazla website tanımlayın).
- Default error page'lerini ayarlayın.
- İstemci request body'si için maksimum izin verilen boyutu belirleyin.
- Bir website için bir URL/route üzerinde kural veya konfigürasyonlar belirtin (burada regex gerekli değildir). Aşağıdakiler arasında: ◦ Route için kabul edilen HTTP method'ları listesi. ◦ HTTP redirection. ◦ İstenen dosyanın bulunması gereken dizin (örneğin, /kapouet URL'si /tmp/www'ye root'lenirse, /kapouet/pouic/toto/pouet URL'si /tmp/www/pouic/toto/pouet'i arayacaktır). ◦ Directory listing'i etkinleştirme veya devre dışı bırakma. ◦ İstenen kaynak bir dizin olduğunda sunulacak default dosya. ◦ İstemcilerden server'a dosya yükleme yetkisi ve depolama konumu sağlanır. ◦ File extension'a göre CGI yürütülmesi (örneğin .php). CGI'lerle ilgili bazı özel açıklamalar:
    - CGI nedir merak mı ediyorsunuz?
    - Web server-CGI iletişiminde yer alan environment variable'ları dikkatlice inceleyin. İstemci tarafından sağlanan tüm request ve argümanlar CGI tarafından kullanılabilir olmalıdır.
    - Chunked request'ler için, server'ınızın bunları un-chunk etmesi gerektiğini unutmayın; CGI EOF'u body'nin sonu olarak bekler.
    - CGI'nin çıktısı için de aynı şey geçerlidir. Eğer CGI'den content_length dönmezse, EOF döndürülen verilerin sonunu işaretleyecektir.
    - CGI, relative path dosya erişimi için doğru dizinde çalıştırılmalıdır.
    - Server'ınız en azından bir CGI'yi desteklemelidir (php-CGI, Python, vb.).

Değerlendirme sırasında her özelliğin çalışır durumda olduğunu test etmek ve göstermek için configuration dosyaları ve default dosyalar sağlamalısınız.

Dosyada diğer kurallar veya configuration bilgileri olabilir (örneğin, virtual host'ları uygulamayı planlıyorsanız bir website için server name).

NOT: Spesifik bir davranış hakkında sorunuz varsa, programınızın davranışını NGINX'inkiyle karşılaştırabilirsiniz.

Küçük bir tester sağladık. Eğer browser'ınız ve testlerinizle her şey iyi çalışıyorsa, onu kullanmak zorunlu değildir; ancak bug'ları bulmanıza ve düzeltmenize yardımcı olabilir.

UYARI: Dayanıklılık anahtardır. Server'ınız her zaman çalışır durumda kalmalıdır.

UYARI: Sadece bir program ile test etmeyin. Testlerinizi daha uygun bir dilde yazın; Python veya Golang gibi (tercih ederseniz C veya C++ de olabilir).

Bölüm V: README Gereksinimleri

Git repository'nizin kök dizininde bir README.md dosyası sağlanmalıdır. Amacı, proje hakkında bilgisi olmayan herhangi bir kişinin (akranlar, personel, recruiter'lar, vb.) projenin ne olduğunu, nasıl çalıştırılacağını ve konu hakkında daha fazla bilgiye nerede bulacağını hızlıca anlamasını sağlamaktır.

README.md en azından aşağıdakileri içermelidir:

- İlk satır italik olmalı ve şu şekilde okunmalıdır: "Bu proje, 42 müfredatının bir parçası olarak <login1>[, <login2>[, <login3>[...]]]] tarafından oluşturulmuştur."
- Projeyi açıkça sunan, hedefini ve kısa bir özeti içeren bir "Açıklama" bölümü.
- Compilation, installation ve/veya execution hakkında ilgili bilgileri içeren bir "Talimatlar" bölümü.
- Konuyla ilgili klasik referansları (dokümantasyon, makaleler, öğreticiler, vb.) listeleyen ve ayrıca AI'nin nasıl kullanıldığını açıklayan (hangi görevler ve projenin hangi kısımları için) bir "Kaynaklar" bölümü.

Projeye bağlı olarak ek bölümler gerekebilir (örneğin, kullanım örnekleri, özellik listesi, teknik seçimler, vb.).

Herhangi bir gerekli ek açıkça aşağıda listelenecektir.

README İngilizce yazılmalıdır.

Bölüm VI: Bonus Kısım

Uygulayabileceğiniz bazı ek özellikler şunlardır:

- Cookie'ler ve session management'ı destekleyin (basit örnekler sağlayın).
- Birden fazla CGI türünü işleyin.

UYARI: Bonus kısım, yalnızca zorunlu kısım tam ve sorunsuz şekilde tamamlanırsa değerlendirilecektir. Tüm zorunlu gereksinimleri karşılamayı başarısız olursanız, bonus kısmınız değerlendirilmeyecektir.

Bölüm VII: Gönderim ve Eş Değerlendirme

Assignment'ınızı Git repository'nize her zamanki gibi gönderin. Savunma sırasında yalnızca repository'nizin içeriği değerlendirilecektir. Dosyalarınızın adlarının doğru olduğundan emin olmak için çift kontrol edin.

Değerlendirme sırasında, projenin kısa bir değişikliği ara sıra talep edilebilir. Bu, küçük bir davranış değişikliği, yazılması veya yeniden yazılması gereken birkaç kod satırı veya kolay eklenebilir bir özellik içerebilir.

Bu adım her projede uygulanmayabilse de, değerlendirme yönergelerinde belirtilmişse buna hazırlı olmalısınız.

Bu adım, projenin belirli bir kısmı hakkında gerçek anlayışınızı doğrulamak için tasarlanmıştır. Değişiklik, seçtiğiniz herhangi bir development ortamında gerçekleştirilebilir

(örneğin, olağan kurulumunuz) ve birkaç dakika içinde mümkün olmalıdır — belirli bir zaman çerçevesi değerlendirmenin bir parçası olarak tanımlanmadıkça.

Örneğin, bir function veya script'de küçük bir güncelleme yapması, bir display değiştirmesi veya yeni bilgi depolamak için bir data structure ayarlaması istenebilirsiniz, vb.

Ayrıntılar (kapsam, hedef, vb.), evaluation guidelines'de belirtilecek ve aynı proje için bir değerlendirmeden diğerine değişebilir.